topic_5_mvc_mvvm_patterns

Theory, Edge Cases & Code Examples

- ✓ What are MVC, MVVM, and MVP?
- ✓ Simple Explanation: Restaurant Analogy
- ✓ MVC Structure
- ✓ MVC Components
- ✓ Model (Data + Logic)

→ Swipe to learn →

topic_5_mvc_mvvm_patterns

Theory

📌 What are MVC, MVVM, and MVP?

These are **architectural patterns** specifically designed for **Graphical User Interface (GUI) applications**. They all aim to **separate UI from business logic**, but do it in slightly different ways.

Core Idea: Don't mix UI code with business logic. Separate "what to display" from "how to display it".

---

topic_5_mvc_mvvm_patterns

Theory

📌 Simple Explanation: Restaurant Analogy

Imagine a restaurant:

Component	Role	Restaurant Analogy
Model	Data + Business Logic	**Kitchen** (prepares food, knows recipes)
View	UI Display	**Menu** (shows options to customer)
Controller/ViewModel/Presenter	Coordinator	**Waiter** (takes orders, brings food)

Flow:
1. Customer looks at **Menu** (View)
2. Customer tells **Waiter** what they want (Controller receives input)
3. Waiter tells **Kitchen** to cook (Controller updates Model)
4. Kitchen prepares food (Model processes data)
5. Waiter brings food to customer (Controller updates View)

Key Insight: Customer (user) never talks directly to Kitchen (Model). Waiter (Controller) coordinates everything.

MVC (Model-View-Controller)

topic_5_mvc_mvvm_patterns

Theory

📌 MVC Structure

```
┌──────────────┐
│    View      │  ← Displays UI (buttons, text, etc.)
│  (UI Layer)  │
└──────────────┘

      ↕ (user input / update)

┌──────────────┐
│ Controller   │  ← Handles user actions, coordinates
│ (Mediator)   │
└──────────────┘

      ↕ (read/write)

┌──────────────┐
│    Model     │  ← Data + Business Logic
│ (Data Layer) │
└──────────────┘
```

topic_5_mvc_mvvm_patterns

Theory

📌 MVC Components

topic_5_mvc_mvvm_patterns

Theory

📌 1. Model (Data + Logic)

What it does:
- Stores **data** (e.g., user info, product list)
- Contains **business logic** (e.g., validation, calculations)
- **Notifies** observers when data changes

What it does NOT do:
- ❌ Know about UI (buttons, labels, etc.)
- ❌ Handle user input

Example:

```
class UserModel {
    std::string name;
    std::string email;
    int age;

    std::vector<std::function<void()>> observers;  // Observer pattern

public:
    void setName(const std::string& newName) {
        name = newName;
        notifyObservers();  // Notify View that data changed
    }

    std::string getName() const { return name; }

    void registerObserver(std::function<void()> observer) {
        observers.push_back(observer);
    }

private:
    void notifyObservers() {
        for (auto& observer : observers) {
            observer();
        }
    }
};
```

---

topic_5_mvc_mvvm_patterns

Theory

📌 2. View (UI Display) (1/2)

What it does:
- **Displays** data to user (labels, text fields, buttons)
- **Sends** user actions to Controller (button clicks, form submissions)
- **Updates** itself when Model changes (via Observer pattern)

What it does NOT do:
- ❌ Business logic
- ❌ Directly modify Model

Example:

```
class UserView {
    UserModel* model;

    // UI Elements (simplified)
    std::string nameLabel;
    std::string emailLabel;

public:
    UserView(UserModel* m) : model(m) {
        // Register as observer
        model->registerObserver([this]() {
            this->updateDisplay();
        });

        updateDisplay();
    }

    void updateDisplay() {
        nameLabel = "Name: " + model->getName();
        emailLabel = "Email: " + model->getEmail();

        std::cout << nameLabel << "\n";
        std::cout << emailLabel << "\n";
    }

    void onButtonClick() {
        // User clicked button → send to Controller
        // (In real app, this would be connected to controller)
// ...continues on next slide
```

📌 2. View (UI Display) (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    }
};
```

topic_5_mvc_mvvm_patterns

Theory

📌 3. Controller (Coordinator)

What it does:
- **Handles** user input from View
- **Updates** Model based on input
- **Selects** which View to show

What it does NOT do:
- ❌ Business logic (that's in Model)
- ❌ UI rendering (that's in View)

Example:

```
class UserController {
    UserModel* model;
    UserView* view;

public:
    UserController(UserModel* m, UserView* v) : model(m), view(v) {}

    void updateUserName(const std::string& newName) {
        // Validation (could also be in Model)
        if (newName.empty()) {
            std::cerr << "Error: Name cannot be empty\n";
            return;
        }

        // Update Model
        model->setName(newName);

        // View automatically updates via Observer pattern!
    }

    void handleFormSubmission(const std::string& name, const std::string& email) {
        // Process user input
        model->setName(name);
        model->setEmail(email);

        // View updates automatically
    }
};
```

---

topic_5_mvc_mvvm_patterns

Theory

📌 MVC Flow

```
1. User interacts with View (clicks button)
        ↓
2. View notifies Controller ("button clicked!")
        ↓
3. Controller updates Model (calls model.setName())
        ↓
4. Model changes data, notifies observers
        ↓
5. View receives notification, updates display
```

Example Flow: User Profile Update

```
User types "Alice" in name field
     ↓
View detects input change
     ↓
View calls controller.updateUserName("Alice")
     ↓
Controller validates input
     ↓
Controller calls model.setName("Alice")
     ↓
Model updates name, calls notifyObservers()
     ↓
View (observer) receives notification
     ↓
View calls updateDisplay()
     ↓
Label now shows "Name: Alice"
```

---

topic_5_mvc_mvvm_patterns

Theory

📌 MVC Benefits

Benefit	Explanation
Separation of Concerns	UI separate from logic
Testability	Can test Model without UI
Multiple Views	One Model, many Views (desktop + mobile)
Reusability	Model can be reused across apps
Parallel Development	Frontend/backend teams work independently

---

topic_5_mvc_mvvm_patterns

Theory

📌 MVC Real-World Examples

Web Frameworks:
- **Django** (Python): Model = Django models, View = templates, Controller = views.py
- **Ruby on Rails**: Model = ActiveRecord, View = ERB templates, Controller = controllers
- **Spring MVC** (Java): @Controller, @Service, Thymeleaf templates

Desktop Applications:
- **Cocoa (macOS/iOS)**: UIViewController, UIView, Data models
- **Qt**: QAbstractItemModel, QWidget, QObject signals/slots

MVVM (Model-View-ViewModel)

topic_5_mvc_mvvm_patterns

Theory

📌 What's Different from MVC?

MVC: Controller sits between View and Model
MVVM: **ViewModel** sits between View and Model, with **two-way data binding**

```
MVC:
View ↔ Controller ↔ Model

MVVM:
View ↔ ViewModel ↔ Model
      (data binding)
```

topic_5_mvc_mvvm_patterns

Theory

📌 MVVM Structure

```
┌──────────────┐
│    View      │  ← UI (XAML, HTML, etc.)
│  (Passive)   │
└──────────────┘

      ↕ (two-way data binding)

┌──────────────┐
│  ViewModel   │  ← Adapter for View
│ (View Logic) │     Exposes data + commands
└──────────────┘

      ↕ (read/write)

┌──────────────┐
│    Model     │  ← Data + Business Logic
│ (Data Layer) │
└──────────────┘
```

topic_5_mvc_mvvm_patterns

Theory

📌 Key Concept: Data Binding

Data Binding: Automatic synchronization between View and ViewModel.

```
XAML (View):
<TextBox Text="{Binding UserName}" />
          ↕ (automatic sync)
C++ (ViewModel):
std::string userName;  // Changes here auto-update UI!
```

When user types in TextBox:
- UI updates ViewModel automatically (no explicit code!)

When code changes `userName`:
- UI updates automatically (no explicit code!)

---

topic_5_mvc_mvvm_patterns

Theory

📌 MVVM Components

topic_5_mvc_mvvm_patterns

Theory

📌 1. Model (Same as MVC)

Data + business logic. No knowledge of UI.

```
class UserModel {
    std::string name;
    std::string email;

public:
    void save() {
        // Save to database
    }
};
```

---

topic_5_mvc_mvvm_patterns

Theory

📌 2. ViewModel (Adapter for View) (1/2)

What it does:
- **Exposes** data for View to bind to
- **Exposes** commands for View to call (button clicks)
- **Transforms** Model data for display (e.g., formatting dates)
- **No direct reference to View** (loose coupling)

What it does NOT do:
- ❌ UI-specific code (no buttons, colors, etc.)
- ❌ Business logic (that's in Model)

Example:

```
class UserViewModel {
    UserModel* model;

    // Observable properties (with change notification)
    ObservableProperty<std::string> userName;
    ObservableProperty<std::string> userEmail;
    ObservableProperty<bool> isFormValid;

public:
    UserViewModel(UserModel* m) : model(m) {
        // Initialize from Model
        userName.set(model->getName());
        userEmail.set(model->getEmail());

        // Watch for changes
        userName.onChange([this]() {
            validateForm();
        });
    }

    // Property getters (for View binding)
    ObservableProperty<std::string>& getUserName() { return userName; }
    ObservableProperty<std::string>& getUserEmail() { return userEmail; }
    ObservableProperty<bool>& getIsFormValid() { return isFormValid; }

    // Commands (for button clicks)
// ...continues on next slide
```

📌 2. ViewModel (Adapter for View) (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    void saveCommand() {
        // Update Model
        model->setName(userName.get());
        model->setEmail(userEmail.get());
        model->save();
    }

private:
    void validateForm() {
        bool valid = !userName.get().empty() && userEmail.get().find('@') != std::string::npos;
        isFormValid.set(valid);
    }
};
```

topic_5_mvc_mvvm_patterns

Theory

📌 3. View (Declarative UI)

What it does:
- **Binds** to ViewModel properties
- **Triggers** ViewModel commands on user actions
- **Purely declarative** (minimal code-behind)

Example (XAML-style):

```
<Window>
    <StackPanel>
        <!-- Two-way binding -->
        <TextBox Text="{Binding UserName, Mode=TwoWay}" />
        <TextBox Text="{Binding UserEmail, Mode=TwoWay}" />

        <!-- Button command binding -->
        <Button Command="{Binding SaveCommand}"
                IsEnabled="{Binding IsFormValid}">
            Save
        </Button>
    </StackPanel>
</Window>
```

Key: View has ZERO logic. Everything is bindings!

---

topic_5_mvc_mvvm_patterns

Theory

📌 MVVM Flow

```
1. User types in TextBox
        ↓
2. Data binding updates ViewModel.userName automatically
        ↓
3. ViewModel.userName.onChange() triggers validation
        ↓
4. ViewModel.isFormValid updates
        ↓
5. Data binding updates Button.IsEnabled automatically
```

No explicit UI update code! Everything is automatic via bindings.

---

topic_5_mvc_mvvm_patterns

Theory

📌 MVVM Benefits

Benefit	Explanation
Less boilerplate	No manual UI update code
Testability	ViewModel has NO UI dependency
Designer-friendly	UI designers work on View, devs on ViewModel
Reactive	UI auto-updates when data changes
Reusability	Same ViewModel for different Views

topic_5_mvc_mvvm_patterns

Theory

📌 MVVM Real-World Examples

Desktop:
- **WPF** (Windows Presentation Foundation) - C#
- **UWP** (Universal Windows Platform) - C#

Web:
- **Angular** - TypeScript (two-way binding with `[(ngModel)]`)
- **Vue.js** - JavaScript (two-way binding with `v-model`)
- **Knockout.js** - JavaScript

Mobile:
- **SwiftUI** (iOS) - `@State`, `@Binding`
- **Jetpack Compose** (Android) - Kotlin

MVP (Model-View-Presenter)

topic_5_mvc_mvvm_patterns

Theory

📌 What's Different from MVC and MVVM?

MVC: View talks to Controller, Controller talks to Model
MVVM: View binds to ViewModel (automatic)
MVP: View is **completely passive**, Presenter controls **everything**

```
MVP:
View ← Presenter ← Model
     (one-way)
```

topic_5_mvc_mvvm_patterns

Theory

📌 MVP Structure

```
┌─────────────────┐
│      View       │  ← Completely passive (interface)
│   (Interface)   │     No logic, just displays what Presenter tells it
└─────────────────┘

        ↑ (presenter calls view methods)

┌─────────────────┐
│    Presenter    │  ← All logic here
│  (Controller)   │     Handles user input, updates View
└─────────────────┘

        ↕

┌─────────────────┐
│     Model       │  ← Data + Business Logic
│  (Data Layer)   │
└─────────────────┘
```

topic_5_mvc_mvvm_patterns

Theory

📌 MVP Components

topic_5_mvc_mvvm_patterns

Theory

📌 1. View (Interface)

Key: View is an **interface**, not a concrete class.

```
class IUserView {
public:
    virtual void displayName(const std::string& name) = 0;
    virtual void displayEmail(const std::string& email) = 0;
    virtual void showError(const std::string& message) = 0;
    virtual ~IUserView() = default;
};
```

topic_5_mvc_mvvm_patterns

Theory

📌 2. Presenter (All Logic)

What it does:
- Holds reference to **View interface** (not concrete View)
- Handles **all user input** (View just forwards to Presenter)
- Updates **View explicitly** (calls view methods)

```
class UserPresenter {
    IUserView* view;
    UserModel* model;

public:
    UserPresenter(IUserView* v, UserModel* m) : view(v), model(m) {}

    void onSaveButtonClicked(const std::string& name, const std::string& email) {
        // Validation
        if (name.empty()) {
            view->showError("Name cannot be empty");
            return;
        }

        // Update Model
        model->setName(name);
        model->setEmail(email);
        model->save();

        // Update View
        view->displayName(model->getName());
        view->displayEmail(model->getEmail());
    }

    void loadUser(int userId) {
        model->load(userId);
        view->displayName(model->getName());
        view->displayEmail(model->getEmail());
    }
};
```

📌 3. Model (Same as MVC/MVVM)

---

topic_5_mvc_mvvm_patterns

Theory

📌 MVP Flow

```
1. User clicks button
        ↓
2. View calls presenter.onSaveButtonClicked()
        ↓
3. Presenter validates input
        ↓
4. Presenter updates Model
        ↓
5. Presenter explicitly calls view.displayName(), view.displayEmail()
        ↓
6. View displays data
```

Key Difference: Presenter **explicitly** calls View methods. No observers, no binding.

---

topic_5_mvc_mvvm_patterns

Theory

📌 MVP Benefits

Benefit	Explanation
Testability	View is interface → easy to mock
Clear control flow	Presenter explicitly updates View
No UI framework dependency	Presenter doesn't know about UI framework

---

MVC vs MVVM vs MVP Comparison

Aspect	MVC	MVVM	MVP
View-Logic Communication	View ↔ Controller	View ↔ ViewModel (binding)	View → Presenter (one-way)
View Intelligence	Some logic	Declarative (bindings)	Completely passive (interface)
Update Mechanism	Observer pattern	Two-way data binding	Explicit method calls
Testability	Good	Excellent	Excellent
Boilerplate	Medium	Low (bindings)	High (explicit calls)
Best For	Web apps, traditional desktop	Modern frameworks (WPF, Angular)	Android apps, legacy systems
View Dependency	View knows Model (optional)	View knows ViewModel	View only knows interface

---

topic_5_mvc_mvvm_patterns

Theory

📌 When to Use Each Pattern

Pattern	Use When
MVC	- Web applications (Django, Rails) - Traditional desktop apps - Simple UI with clear flow
MVVM	- Modern UI frameworks (WPF, SwiftUI, Vue) - Need two-way data binding - Complex reactive UIs
MVP	- Android applications - Need maximum testability - Legacy systems without binding support

---

topic_5_mvc_mvvm_patterns

Edge Cases

📌 1. Fat Controllers / ViewModels (1/2)

Problem: Controller/ViewModel becomes a "God Class" with thousands of lines.

Naive Code:

```
class UserController {
    void handleLogin() { /* 200 lines */ }
    void handleRegistration() { /* 300 lines */ }
    void handlePasswordReset() { /* 150 lines */ }
    void handleProfileUpdate() { /* 200 lines */ }
    void handleEmailVerification() { /* 100 lines */ }
    // ... 50 more methods!
};
```

Why It's Bad:
- Hard to maintain (too much in one place)
- Hard to test (need to test everything)
- Poor cohesion (unrelated features mixed)

Solution 1: Split by Feature

```
// One controller per feature
class LoginController { /* ... */ };
class RegistrationController { /* ... */ };
class ProfileController { /* ... */ };
```

Solution 2: Use Services

```
class UserController {
    LoginService loginService;
    RegistrationService registrationService;

    void handleLogin(const Credentials& creds) {
        // Controller is thin, delegates to service
        bool success = loginService.authenticate(creds);
        if (success) {
            view->navigateToHome();
        }
    }
};
```

topic_5_mvc_mvvm_patterns

Edge Cases

📌 1. Fat Controllers / ViewModels (2/2)

Lesson: Keep controllers **thin**. Delegate to services.

---

topic_5_mvc_mvvm_patterns

Edge Cases

📌 2. View-Model Coupling

Problem: View directly accesses Model, bypassing Controller/ViewModel.

Naive Code (MVC):

```
class UserView {
    UserModel* model;  // ← Direct reference!

    void onButtonClick() {
        // ❌ BAD: View directly modifies Model
        model->setName("Alice");
    }
};
```

Why It's Bad:
- Tight coupling (View depends on Model)
- Hard to test (need real Model)
- Validation bypassed (no Controller involved)

Solution: View Only Talks to Controller

```
class UserView {
    UserController* controller;  // ← Only knows Controller

    void onButtonClick() {
        // ✅ GOOD: Go through Controller
        controller->updateUserName("Alice");
    }
};
```

Lesson: **View should never directly access Model.**

---

topic_5_mvc_mvvm_patterns

Edge Cases

📌 3. Data Binding Performance (MVVM) (1/2)

Problem: Binding to large lists or complex computations causes lag.

Naive Code:

```
class ProductListViewModel {
    ObservableCollection<Product> products;  // 10,000 items!

    void filterProducts(const std::string& query) {
        // ❌ BAD: Expensive operation on every keystroke!
        for (auto& product : products) {
            product.isVisible = product.name.find(query) != std::string::npos;
        }
        notifyPropertyChanged("Products");  // Triggers full UI re-render!
    }
};
```

Why It's Bad:
- **Performance**: Filtering 10,000 items on every keystroke
- **UI freezes**: Blocking main thread

Solution 1: Debouncing (Delay Updates)

```
class ProductListViewModel {
    Timer debounceTimer;

    void onSearchTextChanged(const std::string& query) {
        // Cancel previous timer
        debounceTimer.cancel();

        // Start new timer: execute after 300ms of no typing
        debounceTimer.start(300ms, [this, query]() {
            filterProducts(query);
        });
    }
};
```

📌 3. Data Binding Performance (MVVM) (2/2)

Solution 2: Virtualization (Only Render Visible)

```
// Only bind visible items (e.g., 20 items on screen)
// Scroll → load more items dynamically
ObservableCollection<Product> visibleProducts;  // Only 20 items

void onScroll(int scrollPosition) {
    int startIndex = scrollPosition / itemHeight;
    visibleProducts = allProducts.subList(startIndex, startIndex + 20);
}
```

Solution 3: Background Processing

```
void filterProducts(const std::string& query) {
    // Run filtering on background thread
    std::async([this, query]() {
        auto filtered = expensiveFilter(query);

        // Update UI on main thread
        runOnUIThread([this, filtered]() {
            products = filtered;
            notifyPropertyChanged("Products");
        });
    });
}
```

Lesson: **Optimize bindings** for large data sets. Use debouncing, virtualization, or background threads.

topic_5_mvc_mvvm_patterns

Edge Cases

📌 4. Testing Challenges (Tight View Coupling) (1/2)

Problem: Can't test Controller without real View.

Naive Code:

```
class UserController {
    ConcreteUserView* view;  // ← Concrete dependency!

public:
    void updateUser(const std::string& name) {
        model->setName(name);
        view->updateNameLabel();  // ← Requires real UI!
    }
};

// Test
void testUpdateUser() {
    ConcreteUserView view;  // ❌ Need real UI framework!
    UserController controller(&view);
    controller.updateUser("Alice");
}
```

Why It's Bad:
- Can't test without UI framework
- Slow tests (need to create UI components)

📌 4. Testing Challenges (Tight View Coupling) (2/2)

Solution: Depend on Interface (MVP Style)

```
class IUserView {
public:
    virtual void updateNameLabel(const std::string& name) = 0;
    virtual ~IUserView() = default;
};

class UserController {
    IUserView* view;  // ← Interface dependency

public:
    void updateUser(const std::string& name) {
        model->setName(name);
        view->updateNameLabel(name);
    }
};

// Mock View
class MockUserView : public IUserView {
public:
    std::string lastNameUpdate;

    void updateNameLabel(const std::string& name) override {
        lastNameUpdate = name;  // Record for verification
    }
};

// Test
void testUpdateUser() {
    MockUserView mockView;
    UserController controller(&mockView);

    controller.updateUser("Alice");

    assert(mockView.lastNameUpdate == "Alice");  // ✅ No real UI needed!
}
```

Lesson: **Use interfaces** for testability. Mock View in tests.

topic_5_mvc_mvvm_patterns

Edge Cases

📌 5. State Management Complexity (1/3)

Problem: Multiple Views need to share state. Where to store it?

Naive Code:

```
// ❌ BAD: State duplicated across controllers
class ProductListController {
    std::vector<Product> selectedProducts;
};

class CheckoutController {
    std::vector<Product> selectedProducts;  // ← Duplicate!
};

// How to keep these in sync?
```

Why It's Bad:
- State duplication
- Synchronization bugs (one updates, other doesn't)

📌 5. State Management Complexity (2/3)

Solution 1: Shared Model

```
class ShoppingCartModel {  // Singleton or shared instance
    std::vector<Product> selectedProducts;
    std::vector<std::function<void()>> observers;

public:
    void addProduct(const Product& product) {
        selectedProducts.push_back(product);
        notifyObservers();
    }

    const std::vector<Product>& getProducts() const {
        return selectedProducts;
    }

    void registerObserver(std::function<void()> observer) {
        observers.push_back(observer);
    }

private:
    void notifyObservers() {
        for (auto& obs : observers) obs();
    }
};

// Both controllers reference same Model
class ProductListController {
    ShoppingCartModel* cartModel;  // Shared

    void onProductSelected(const Product& product) {
        cartModel->addProduct(product);
    }
};

class CheckoutController {
    ShoppingCartModel* cartModel;  // Same instance

    void displayCart() {
        for (const Product& p : cartModel->getProducts()) {
            // Display
// ...continues on next slide
```

📌 5. State Management Complexity (3/3)

[Code continues across slides]

```
// ...continued from previous slide
        }
    }
};
```

Solution 2: Event Bus (Decoupled)

```
// Controllers don't directly share state
class ProductListController {
    void onProductSelected(const Product& product) {
        eventBus.publish(ProductSelectedEvent{product});
    }
};

class CheckoutController {
    void init() {
        eventBus.subscribe("ProductSelected", [this](const Event& e) {
            selectedProducts.push_back(e.product);
            updateDisplay();
        });
    }
};
```

Lesson: **Centralize shared state** in a Model or use Event Bus for communication.

---

topic_5_mvc_mvvm_patterns

Edge Cases

📌 6. Navigation Logic (Where Does It Go?) (1/3)

Problem: Who is responsible for navigating between Views?

Naive Code:

```
// ❌ Option 1: View navigates (bad - tight coupling)
class LoginView {
    void onLoginSuccess() {
        HomeView* homeView = new HomeView();
        homeView->show();  // ← View creates another View!
    }
};

// ❌ Option 2: Model navigates (bad - Model shouldn't know about UI)
class UserModel {
    void login() {
        if (authenticated) {
            HomeView* homeView = new HomeView();  // ← Model knows about View!
        }
    }
};
```

Why It's Bad:
- **Option 1**: View coupled to other Views
- **Option 2**: Model coupled to UI

📌 6. Navigation Logic (Where Does It Go?) (2/3)

Solution 1: Controller Handles Navigation

```
class LoginController {
    LoginView* loginView;
    Navigator* navigator;

public:
    void onLoginButtonClicked(const Credentials& creds) {
        bool success = model->authenticate(creds);

        if (success) {
            navigator->navigateTo("home");  // ✅ Controller decides
        } else {
            loginView->showError("Invalid credentials");
        }
    }
};

class Navigator {
public:
    void navigateTo(const std::string& destination) {
        if (destination == "home") {
            homeController->show();
        } else if (destination == "profile") {
            profileController->show();
        }
    }
};
```

📌 6. Navigation Logic (Where Does It Go?) (3/3)

Solution 2: ViewModel with Navigation Service

```
class LoginViewModel {
    INavigationService* navigationService;

public:
    void loginCommand(const Credentials& creds) {
        bool success = model->authenticate(creds);

        if (success) {
            navigationService->navigateTo("home");  // ✅ ViewModel uses service
        }
    }
};
```

Lesson: **Controller/ViewModel handles navigation**, using a Navigation service.

---

topic_5_mvc_mvvm_patterns

Code Examples

📌 1. Simple MVC (User Profile) (1/5)

Goal: Implement MVC pattern for displaying and editing user profile.

Output:

```
#include <iostream>
#include <string>
#include <vector>
#include <functional>

// ═══════════════════════════════════════════════════════════════
//  MODEL
// ═══════════════════════════════════════════════════════════════

class UserModel {
    std::string name;
    std::string email;
    int age;

    std::vector<std::function<void()>> observers;

public:
    UserModel(const std::string& n, const std::string& e, int a)
        : name(n), email(e), age(a) {}

    // Getters
    std::string getName() const { return name; }
    std::string getEmail() const { return email; }
    int getAge() const { return age; }

    // Setters (trigger notification)
    void setName(const std::string& n) {
        name = n;
        notifyObservers();
    }

    void setEmail(const std::string& e) {
        email = e;
        notifyObservers();
    }

// ...continues on next slide
```

topic_5_mvc_mvvm_patterns

Code Examples

📌 1. Simple MVC (User Profile) (2/5)

```
// ...continued from previous slide
    void setAge(int a) {
        age = a;
        notifyObservers();
    }

    // Observer pattern
    void registerObserver(std::function<void()> observer) {
        observers.push_back(observer);
    }

private:
    void notifyObservers() {
        std::cout << "[MODEL] Data changed, notifying observers...\n";
        for (auto& obs : observers) {
            obs();
        }
    }
};

// ═══════════════════════════════════════════════════════════════
//  VIEW
// ═══════════════════════════════════════════════════════════════

class UserView {
    UserModel* model;

public:
    UserView(UserModel* m) : model(m) {
        // Register as observer
        model->registerObserver([this]() {
            this->render();
        });

        render();  // Initial render
    }

    void render() {
        std::cout << "\n╔═══════════════════════════════╗\n";
        std::cout << "║      User Profile             ║\n";
        std::cout << "╚═══════════════════════════════╝\n";
// ...continues on next slide
```

📌 1. Simple MVC (User Profile) (3/5)

```
// ...continued from previous slide
        std::cout << "  Name:  " << model->getName() << "\n";
        std::cout << "  Email: " << model->getEmail() << "\n";
        std::cout << "  Age:   " << model->getAge() << "\n";
        std::cout << "═══════════════════════════════\n";
    }

    void simulateUserInput(std::function<void()> callback) {
        // In real app, this would be button click, form submission, etc.
        callback();
    }
};

// ═══════════════════════════════════════════════════════════
//  CONTROLLER
// ═══════════════════════════════════════════════════════════

class UserController {
    UserModel* model;
    UserView* view;

public:
    UserController(UserModel* m, UserView* v) : model(m), view(v) {}

    void updateName(const std::string& newName) {
        std::cout << "\n[CONTROLLER] Updating name to: " << newName << "\n";

        // Validation
        if (newName.empty()) {
            std::cerr << "[CONTROLLER] Error: Name cannot be empty!\n";
            return;
        }

        // Update model (view will auto-update via observer)
        model->setName(newName);
    }

    void updateEmail(const std::string& newEmail) {
        std::cout << "\n[CONTROLLER] Updating email to: " << newEmail << "\n";

        // Validation
// ...continues on next slide
```

topic_5_mvc_mvvm_patterns

Code Examples

📌 1. Simple MVC (User Profile) (4/5)

```
// ...continued from previous slide
        if (newEmail.find('@') == std::string::npos) {
            std::cerr << "[CONTROLLER] Error: Invalid email format!\n";
            return;
        }

        model->setEmail(newEmail);
    }

    void updateAge(int newAge) {
        std::cout << "\n[CONTROLLER] Updating age to: " << newAge << "\n";

        // Validation
        if (newAge < 0 || newAge > 150) {
            std::cerr << "[CONTROLLER] Error: Invalid age!\n";
            return;
        }

        model->setAge(newAge);
    }
};

// ═══════════════════════════════════════════════════════════════
//  MAIN
// ═══════════════════════════════════════════════════════════════

int main() {
    std::cout << "╔═══════════════════════════════════════════════╗\n";
    std::cout << "║  MVC Pattern: User Profile Example           ║\n";
    std::cout << "╚═══════════════════════════════════════════════╝\n";

    // Create MVC components
    UserModel model("Alice", "alice@example.com", 25);
    UserView view(&model);
    UserController controller(&model, &view);

    // Simulate user interactions
    std::cout << "\n=== User updates name ===\n";
    controller.updateName("Alice Smith");

    std::cout << "\n=== User tries invalid name ===\n";
// ...continues on next slide
```

topic_5_mvc_mvvm_patterns

Code Examples

📌 1. Simple MVC (User Profile) (5/5)

[Code continues across slides]

```
// ...continued from previous slide
    controller.updateName("");

    std::cout << "\n=== User updates email ===\n";
    controller.updateEmail("alice.smith@example.com");

    std::cout << "\n=== User tries invalid email ===\n";
    controller.updateEmail("invalid-email");

    std::cout << "\n=== User updates age ===\n";
    controller.updateAge(26);

    std::cout << "\n╔═══════════════════════════════════════════╗\n";
    std::cout << "│  Key Points:                              │\n";
    std::cout << "│  1. Model notifies observers on data change  │\n";
    std::cout << "│  2. View automatically re-renders         │\n";
    std::cout << "│  3. Controller handles validation         │\n";
    std::cout << "╚═══════════════════════════════════════════╝\n";

    return 0;
}
```

```
['╔═══════════════════════════════════════╗\n│  MVC Pattern: User Profile Example          │\n╚═══════════
```

topic_5_mvc_mvvm_patterns

Code Examples

📌 2. MVVM with Observable Properties (1/6)

Goal: Implement MVVM with data binding simulation.

```
#include <iostream>
#include <string>
#include <vector>
#include <functional>

// ═══════════════════════════════════════════════════════════════
//  OBSERVABLE PROPERTY (Data Binding Simulation)
// ═══════════════════════════════════════════════════════════════

template<typename T>
class ObservableProperty {
    T value;
    std::vector<std::function<void(const T&)>> changeListeners;

public:
    ObservableProperty() = default;
    ObservableProperty(const T& val) : value(val) {}

    void set(const T& newValue) {
        if (value != newValue) {
            value = newValue;
            notifyListeners();
        }
    }

    T get() const { return value; }

    void onChange(std::function<void(const T&)> listener) {
        changeListeners.push_back(listener);
    }

private:
    void notifyListeners() {
        for (auto& listener : changeListeners) {
            listener(value);
        }
    }
// ...continues on next slide
```

📌 2. MVVM with Observable Properties (2/6)

```
// ...continued from previous slide
};

// ═══════════════════════════════════════════════════════════════
//  MODEL
// ═══════════════════════════════════════════════════════════════

struct User {
    std::string name;
    std::string email;
    int age;
};

class UserModel {
public:
    User loadUser(int userId) {
        std::cout << "[MODEL] Loading user from database...\n";
        return {"Alice", "alice@example.com", 25};
    }

    void saveUser(const User& user) {
        std::cout << "[MODEL] Saving user to database...\n";
        std::cout << "  Name: " << user.name << "\n";
        std::cout << "  Email: " << user.email << "\n";
        std::cout << "  Age: " << user.age << "\n";
    }
};

// ═══════════════════════════════════════════════════════════════
//  VIEWMODEL
// ═══════════════════════════════════════════════════════════════

class UserViewModel {
    UserModel model;

public:
    // Observable properties (for View binding)
    ObservableProperty<std::string> userName;
    ObservableProperty<std::string> userEmail;
    ObservableProperty<int> userAge;
    ObservableProperty<bool> isFormValid;
// ...continues on next slide
```

📌 2. MVVM with Observable Properties (3/6)

```
// ...continued from previous slide
    ObservableProperty<std::string> statusMessage;

    UserViewModel() {
        // Load initial data
        User user = model.loadUser(1);
        userName.set(user.name);
        userEmail.set(user.email);
        userAge.set(user.age);

        // Setup validation
        userName.onChange([this](const std::string&) { validateForm(); });
        userEmail.onChange([this](const std::string&) { validateForm(); });
        userAge.onChange([this](int) { validateForm(); });

        validateForm();
    }

    // Command: Save button clicked
    void saveCommand() {
        std::cout << "\n[VIEWMODEL] Save command executed\n";

        if (!isFormValid.get()) {
            statusMessage.set("Cannot save: Form has errors");
            return;
        }

        User user = {userName.get(), userEmail.get(), userAge.get()};
        model.saveUser(user);

        statusMessage.set("User saved successfully!");
    }

private:
    void validateForm() {
        std::cout << "[VIEWMODEL] Validating form...\n";

        bool valid = true;
        std::string message = "";

        if (userName.get().empty()) {
// ...continues on next slide
```

📌 2. MVVM with Observable Properties (4/6)

```
// ...continued from previous slide
            valid = false;
            message = "Name is required";
        } else if (userEmail.get().find('@') == std::string::npos) {
            valid = false;
            message = "Invalid email format";
        } else if (userAge.get() < 0 || userAge.get() > 150) {
            valid = false;
            message = "Invalid age";
        } else {
            message = "Form is valid";
        }

        isFormValid.set(valid);
        statusMessage.set(message);
    }
};

// ═══════════════════════════════════════════════════════════════
//  VIEW (Simulated)
// ═══════════════════════════════════════════════════════════════

class UserView {
    UserViewModel* viewModel;

public:
    UserView(UserViewModel* vm) : viewModel(vm) {
        setupBindings();
        render();
    }

    void setupBindings() {
        std::cout << "[VIEW] Setting up data bindings...\n";

        // Bind to ViewModel properties
        viewModel->userName.onChange([this](const std::string&) { render(); });
        viewModel->userEmail.onChange([this](const std::string&) { render(); });
        viewModel->userAge.onChange([this](int) { render(); });
        viewModel->isFormValid.onChange([this](bool) { render(); });
        viewModel->statusMessage.onChange([this](const std::string&) { render(); });
    }
// ...continues on next slide
```

📌 2. MVVM with Observable Properties (5/6)

```
// ...continued from previous slide

    void render() {
        std::cout << "\n╔═══════════════════════════════════════╗\n";
        std::cout << "│      User Profile Form (MVVM)         │\n";
        std::cout << "└═══════════════════════════════════════╝\n";
        std::cout << "  Name:  [" << viewModel->userName.get() << "]\n";
        std::cout << "  Email: [" << viewModel->userEmail.get() << "]\n";
        std::cout << "  Age:   [" << viewModel->userAge.get() << "]\n";
        std::cout << "  \n";
        std::cout << "  [Save Button] " << (viewModel->isFormValid.get() ? "Enabled" : "Disabled") << "\n";
        std::cout << "  \n";
        std::cout << "  Status: " << viewModel->statusMessage.get() << "\n";
        std::cout << "═══════════════════════════════════════\n";
    }

    void simulateUserInput(const std::string& field, const std::string& value) {
        std::cout << "\n[VIEW] User typed in " << field << " field: " << value << "\n";

        if (field == "name") {
            viewModel->userName.set(value);
        } else if (field == "email") {
            viewModel->userEmail.set(value);
        }
    }

    void simulateButtonClick() {
        std::cout << "\n[VIEW] User clicked Save button\n";
        viewModel->saveCommand();
    }
};

// ═══════════════════════════════════════════════════════════
//  MAIN
// ═══════════════════════════════════════════════════════════

int main() {
    std::cout << "╔═══════════════════════════════════════╗\n";
    std::cout << "│  MVVM Pattern: Data Binding Example   │\n";
    std::cout << "└═══════════════════════════════════════╝\n\n";

// ...continues on next slide
```

📌 2. MVVM with Observable Properties (6/6)

[Code continues across slides]

```
// ...continued from previous slide
    UserViewModel viewModel;
    UserView view(&viewModel);

    // Simulate user interactions
    view.simulateUserInput("name", "Alice Smith");
    view.simulateUserInput("email", "alice.smith@example.com");
    view.simulateButtonClick();

    std::cout << "\n--- User tries invalid email ---\n";
    view.simulateUserInput("email", "invalid");

    std::cout << "\n╔═══════════════════════════════════════╗\n";
    std::cout << "║  Key Points:                           ║\n";
    std::cout << "║  1. View binds to ViewModel properties  ║\n";
    std::cout << "║  2. Changes auto-trigger UI updates     ║\n";
    std::cout << "║  3. No direct Model reference in View   ║\n";
    std::cout << "╚═══════════════════════════════════════╝\n";

    return 0;
}
```

What You Learned

Key Takeaways

- ✓ What are MVC, MVVM, and MVP?
- ✓ Simple Explanation: Restaurant Analogy
- ✓ MVC Structure
- ✓ MVC Components

→ Swipe to learn →